

前言

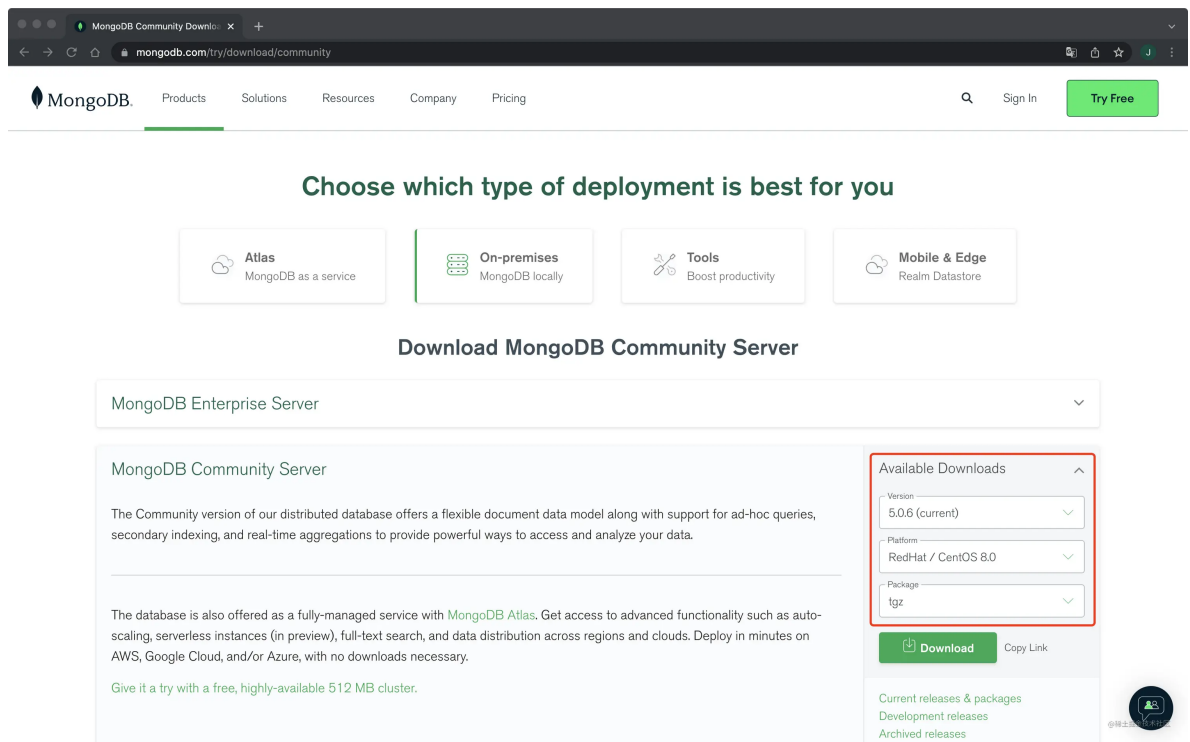
有了 Node 的加持，可在服务端使用 JS 编写增删改查的接口应用。既然涉及接口应用，那肯定得有一个数据库作为支撑。很多业务场景都会选择关系型数据库，而关系型数据库的代表就是 MySQL。作为一位资深的开发者，我认为 MongoDB 更适合在前端领域中管理数据。

虽然现在关系型数据库还是主流，但面对某些需求时，需非关系型数据库作为补充，学习一个主流 NoSQL 数据库 还是很有必要的。MongoDB 是一个功能丰富的 NoSQL 数据库，其使用 JSON 管理数据，就像 JS 操作 JSON 一样方便，无需事先设置数据结构且字段可随时设置，要是用于保存 Restful API 请求 与 返回 JSON 数据 就更方便了，无论数据有多复杂，直接像处理 JSON 一样处理数据即可。

发现很多公司都会寻求阿里云、腾讯云、百度云、华为云等第三方平台提供付费的数据库，但我觉得无必要，自食其力搭建一个 MongoDB 不是更好吗？本章将带领你不花一分钱部署 NoSQL 数据库，结合 mongoose 像 JS 操作 JSON 一样操作数据库，同时部署到服务器并使用接口工具完成数据调试流程。

搭建：从零到一搭建数据库

打开 MongoDB 的[下载地址](#)，选择以下配置再下载。打开浏览器下载列表，查看当前文件的下载的地址为 https://fastdl.mongodb.org/linux/mongodb-linux-x86_64-rhel80-5.0.6.tgz，再取消当前的下载。



安装

打开 **CMD工具**，登录服务器。因为 **yum** 不存在 **Mongodb源**，所以无法执行 **yum install mongodb** 安装 **MongoDB**。通过以下方式手动安装 **MongoDB**。

进入工具目录

```
cd /tool
```

下载MongoDB

```
wget https://fastdl.mongodb.org/linux/mongodb-linux-x86_64-rhel80-5.0.6.tgz
```

解压MongoDB

```
tar -zxvf mongodb-linux-x86_64-rhel80-5.0.6.tgz -C /tool
```

重命名目录

```
mv mongodb-linux-x86_64-rhel80-5.0.6 mongodb
```

配置

安装完毕还需手动增加一些配置，为了方便管理 MongoDB 相关文件，将这些配置文件都存放到 `mongodb` 文件夹中。

进入目录并创建文件夹与日志文件

```
cd mongodb && mkdir data && mkdir log && touch log/mongodb.log
```

生效环境变量

设置软链接，可通过 `mongod` 快速调用命令。

```
echo "export PATH=$PATH:/tool/mongodb/bin" >> ~/.bash_profile
source ~/.bash_profile
```

执行 `vim /tool/mongodb/mongodb.conf`，加入以下内容。该文件作为 MongoDB 的配置文件，在执行 `mongod` 时必须指定配置文件的路径。

```
# 数据库
dbpath=/tool/mongodb/data
# 日志文件
logpath=/tool/mongodb/log/mongodb.log
# 使用追加的方式更新日志
logappend=true
# 端口
port=27017
# 以守护进程的方式运行MongoDB(创建服务器进程)
fork=true
# 启用用户验证
# auth=true
# 绑定服务IP(绑定127.0.0.1只能本机访问，若不指定则默认本地所有IP)
bind_ip=0.0.0.0
```

执行 `mongod -f /tool/mongodb/mongodb.conf` 启动 MongoDB，输出以下信息表示开启成功。

```
about to fork child process, waiting until server is ready for connections.
forked process: 2028
child process started successfully, parent exiting
```

执行 `ps -ef | grep mongod` 查看 MongoDB 状态，输出以下信息表示 MongoDB 正常运行。

```
root      2028      1  0 22:44 ?          00:00:02 mongod -f /tool/mongodb/mongodb.conf
root      2281    2072  0 22:48 pts/3    00:00:00 grep --color=auto mongod
```

执行 `mongod --shutdown -f /tool/mongodb/mongodb.conf` 关闭 MongoDB，输出以下信息表示关闭成功。

```
killing process with pid: 2028
```

若无使用 MongoDB 的应用在运行，请关闭 MongoDB，这样才能让服务器在 CPU 低占用的情况下保持稳定的性能。当用到 MongoDB 时再开启也不迟。

连接

在启动 MongoDB 的情况下，执行 `mongo` 连接 MongoDB，输出以下信息表示连接成功。

```
MongoDB shell version v5.0.6
connecting to: mongodb://127.0.0.1:27017/?compressors=disabled&gssapiServiceName=mongod
Implicit session: session { "id" : UUID("a1cb8b2e-3ac2-4be9-8547-eb161c4dea74") }
MongoDB server version: 5.0.6
```

连接 MongoDB 后，终端会进入 MongoDB 模式，该模式可执行 MongoDB 相关命令，可查看[MongoDB教程](#)，在此不深入讲述了。

执行 `use admin` 切换到 admin 数据库，该数据库是 MongoDB 的默认数据库，用于管理用户权限。在上述配置文件中并未开启 `auth=true`，因此创建 root 用户后需开

启 `auth=true` 。

执行以下命令为 `admin` 数据库创建 `root` 用户。

```
db.createUser({ user: "root", pwd: "123456", roles: [{ role: "root", db: "admin" }] })
```

输出以下信息表示创建成功。

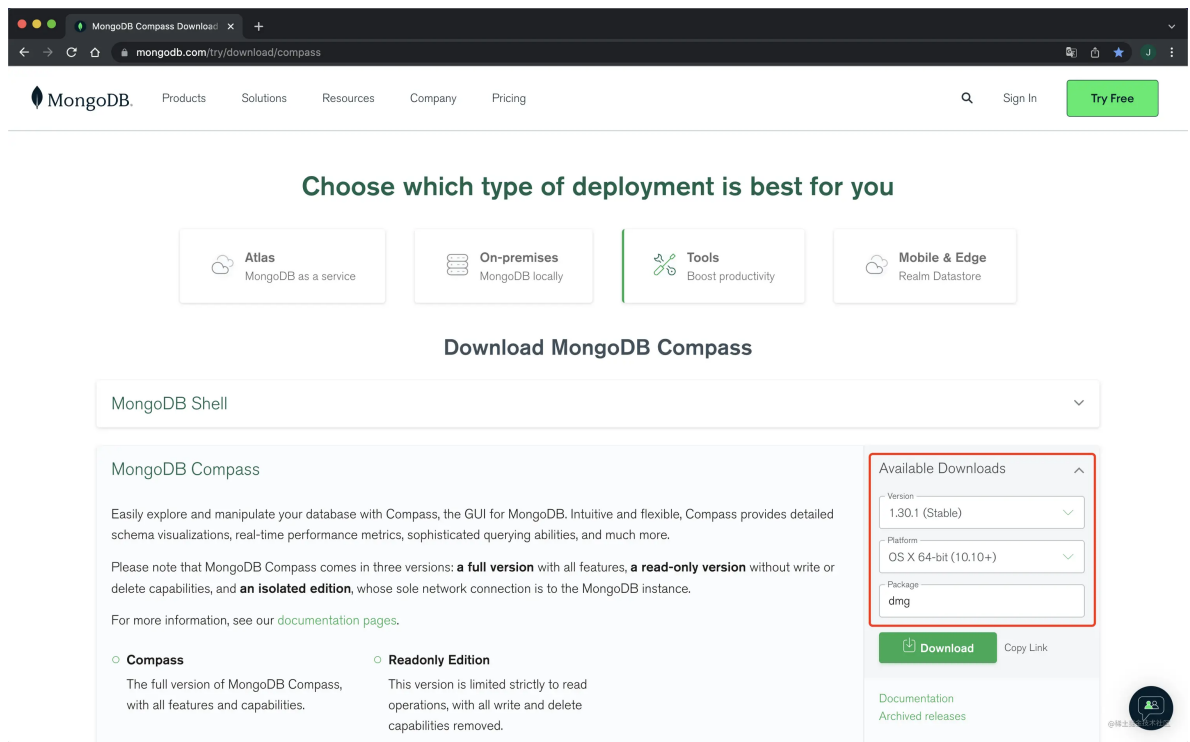
```
Successfully added user: {
  "user": "root",
  "roles": [{
    "role": "root",
    "db": "admin"
  }]
}
```

执行 `vim /tool/mongodb/mongodb.conf` 将 `# auth=true` 的 `#` 去掉，重启 MongoDB 。
重启动作包括 `mongod -f /tool/mongodb/mongodb.conf` 与 `mongod --shutdown -f /tool/mongodb/mongodb.conf` 。

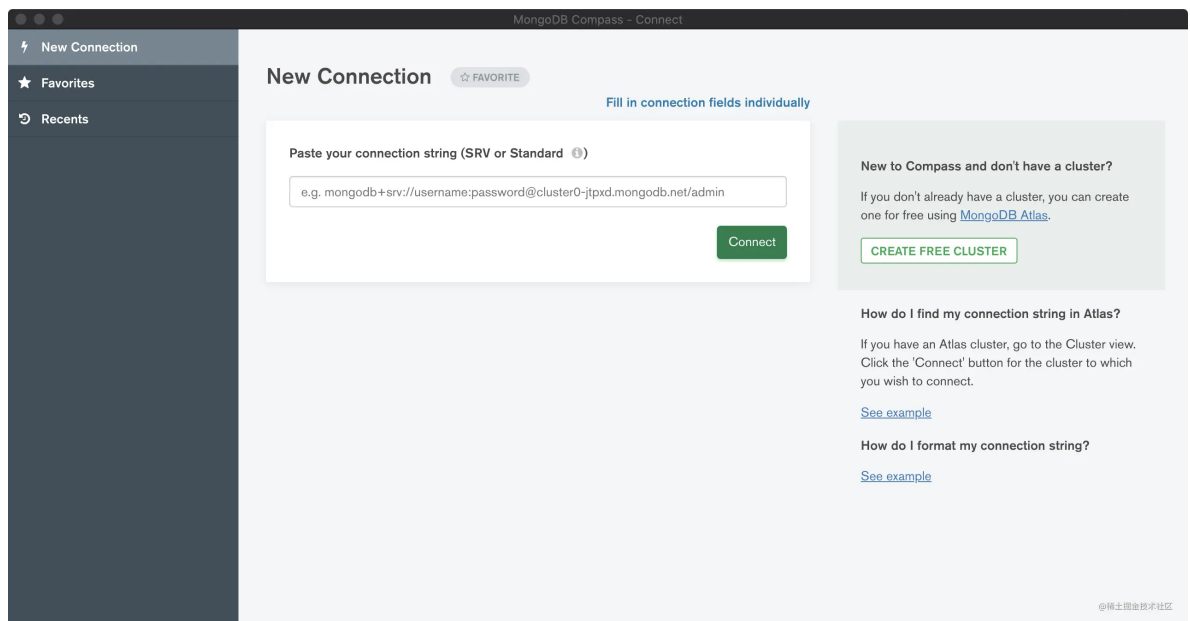
执行以下命令，输出 `1` 表示用户登录成功。接着可基于 MongoDB 操作数据了。

```
use admin
db.auth("root", "123456")
```

在不熟悉 MongoDB 命令 的情况下，盲目操作数据库会引发很大风险，因此不建议在 CMD 工具 中操作 MongoDB 。好在 MongoDB 官方 提供一个界面漂亮操作流畅的 GUI 工具 MongoDB Compass ，不习惯命令操作 MongoDB 的同学可用 Compass 。打开 Compass 的[下载地址](#)，选择以下配置再下载。



打开 **Compass** , 发现需填写一条 **MongoDB标准URL** 才能连接数据库。



MongoDB标准URL 是什么？它是 **MongoDB** 基于 **shell命令** 连接数据库的 **标准化URL** , 由以下参数组成。

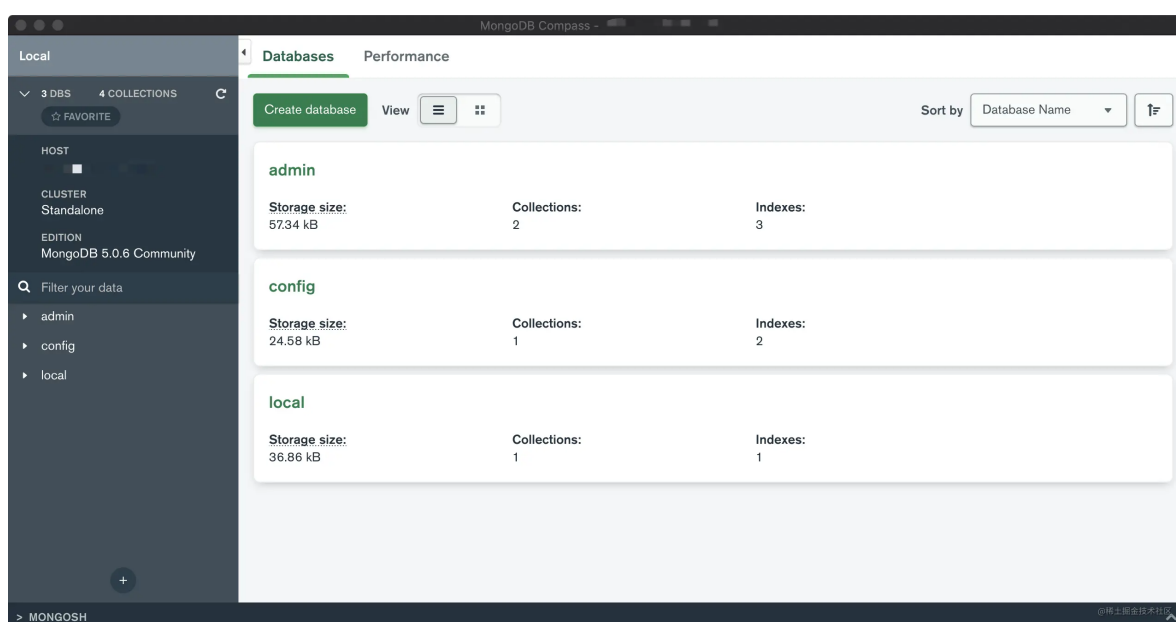
```
mongodb://username:password@host:port/database[?options]
```

txt

参数	说明	描述
mongodb://	协议	可理解成 HTTP
username	账号	上述创建的 root
password	密码	上述创建的 root 的密码 123456
host	实例公有IP	云服务器IP
port	端口	默认 27017
database	数据库	上述切换的 admin 数据库
options	配置	用得很少

那拼接起来的 URL 就是 `mongodb://root:123456@aaa.bbb.ccc.ddd:27017/admin`，输入到 Compass 中就可连接数据库了。首次连接肯定失败，那是因为还未配置服务器的安全组。

每次新开端口都需在服务器中配置安全组，可回看第7章的 [个人官网-安全组配置](#) 那部分内容。这次增加 27017 端口，那在安全组中配置一个 27017 端口。再次连接 MongoDB 就可正常访问了。



开发：初试小型接口系统

自行部署数据库相比选购第三方平台数据可省下好几千元。因为 MongoDB 文档 的概念与 JSON 很相似，所以很多开发者都会把 MongoDB 作为首选数据库。

为了让 MongoDB 能派上用场，以下会在 Node 环境中结合 mongoose 开发一个小型接口系统，通过 Nginx 代理接口，输出一个可基于 HTTPS 访问的接口系统。设计该场景的目的是为了让你掌握以下技能，通过 数据库、数据操作 和 接口调试 三个突破点学习 接口系统 的搭建思路。当然代码内容不是主要，主要的是搭建思路的大体框架。



在本地环境创建项目 data-base，使用 koa 创建 Node 服务，该服务启动后初始全部接口的路由实例，每个路由实例根据需求处理数据并使用 mongoose 读写数据库。

该服务提供一个产品的增删改查功能，整体围绕着产品数据做读写操作。

初始项目

执行 npm init 创建 package.json，执行以下命令安装项目所需依赖。

```
npm i @yangzw/bruce-us cross-env dayjs koa koa-body koa-json koa-logger koa-onerror koa-  
npm i nodemon -D
```

启动服务

在根目录中创建 `src` 文件夹，在该文件夹中创建入口文件 `index.js`。该文件使用 `koa` 创建 `Node`服务，接入一些常见中间件升级 `koa` 的功能。

```
import Koa from "koa";
import KoaBody from "koa-body";
import KoaJson from "koa-json";
import KoaLogger from "koa-logger";
import KoaOnerror from "koa-onerror";

// 创建实例
const app = new Koa();
KoaOnerror(app); // 美化错误参数
app.on("error", (err, ctx) => console.error("server error", err, ctx)); // 捕获错误

// 配置中间件
app.use(KoaLogger()); // 日志解析
app.use(KoaBody({ multipart: true })); // Body解析
app.use(KoaJson()); // JSON解析

// 监听服务
app.listen(3000);
console.log("Node服务已启动，监听端口3000");
```

[点击查看源码。](#)

这样一个基础的 `Node`服务 就搭建好了。在 `package.json` 中指定 `scripts`，加入开发环境与生产环境的启动命令。在开发环境中为了修改代码后能重启 `Node`服务，使用 `nodemon` 代理 `node` 的启动功能，可回看第2章的 `监听脚本自动重启命令` 那部分内容。

```
{
  "scripts": {
    "clean": "rimraf coverage dist node_modules package-lock.json yarn.lock"
    "deploy": "cross-env NODE_ENV=prod node --es-module-specifier-resolutior
    "start": "cross-env NODE_ENV=dev nodemon --es-module-specifier-resolutic
  }
}
```

[点击查看源码。](#)

执行 `npm start`，输出以下信息表示启动成功。

Node服务已启动，监听端口3000

连接数据库

有了上述搭建好的 MongoDB 作为载体，整个 接口系统 在执行时产生的数据都经过 MongoDB 处理，处理 MongoDB数据 当然离不开 mongoose 。

mongoose是一个在 Node环境 中操作 MongoDB对象模型 的 Npm模块，其封装了 MongoDB 对文档增删改查的方法。其文档已很完善，可查看[mongoose文档](#)，在此不深入讲述了。

还记得 MongoDB标准URL 的组成吗？连接数据库需用到它。

```
mongodb://username:password@host:port/database[?options]
```

txt

为了方便管理这些参数，在 src 文件夹中创建 app.config.js 文件。在本地环境使用 实例公有IP，在服务器环境使用 127.0.0.1。

```
import { env } from "process";

export default {
  mongodb: {
    host: env.NODE_ENV === "dev" ? "your server host" : "127.0.0.1",
    password: "123456",
    port: 27017,
    username: "root"
  }
};
```

js

点击查看[源码](#)。

在 src 文件夹中创建 database/index.js 文件。使用 connect() 连接数据库，使用 connection() 监听数据库状态事件。

```
import Mongoose from "mongoose";

import AppConfig from "../app.config";

const { connect, connection } = Mongoose;
```

js

```
const { mongodb: { host, password, port, username } } = AppConfig;

connect(`mongodb://${username}:${password}@${host}:${port}/mall`, {
  authSource: "admin",
  useNewUrlParser: true,
  useUnifiedTopology: true
});

connection.on("connected", () => console.log("数据库连接成功"));
connection.on("disconnected", () => console.log("数据库连接断开"));
connection.on("error", () => console.log("数据库连接异常"));
```

点击查看[源码](#)。

保存文件后会自动重启 **Node服务**，输出以下信息表示启动成功。

```
Node服务已启动，监听端口3000
数据库连接成功
```

定义数据

mongoose 中有三个很重要的概念，分别是 **Schema**、**Model** 和 **Entity**。

- **Schema**: 表示模式，一种以文档形式存储的数据库模型骨架，不具备数据库操作能力
- **Model**: 表示模型，由 **Schema** 生成的模型，具备抽象属性与数据库操作能力
- **Entity**: 表示实例，由 **Model** 创建的实例，具备操作数据库操作能力

mongoose 中任何事物都要从 **Schema** 开始。每个 **Schema** 对应 **MongoDB** 中的一个集合 **Collection**。**Schema** 中定义了集合中文档 **Document** 的格式。

通过 **Schema** 为产品数据定义一个模式，再通过 **Model** 为产品模式定义一个模型。

```
import Day from "dayjs";
import Mongoose from "mongoose";

const { model, Schema } = Mongoose;

// 定义模式
const Product = {
  brand: {
    match: /^.{2,200}$/,
```

js

```

        msg: "品牌只能由2到200位任意字符组成",
        required: true,
        trim: true,
        type: String
    },
    code: {
        match: /^[A-Za-z0-9]{4,30}$/,
        msg: "条形码只能由4到30位英文或数字组成",
        required: true,
        trim: true,
        type: String
    },
    createtime: { ... },
    description: { ... },
    name: { ... }
    origin: { ... }
};
// 定义模型
const ProductModel = model("product", new Schema(Product, { versionKey: false }));

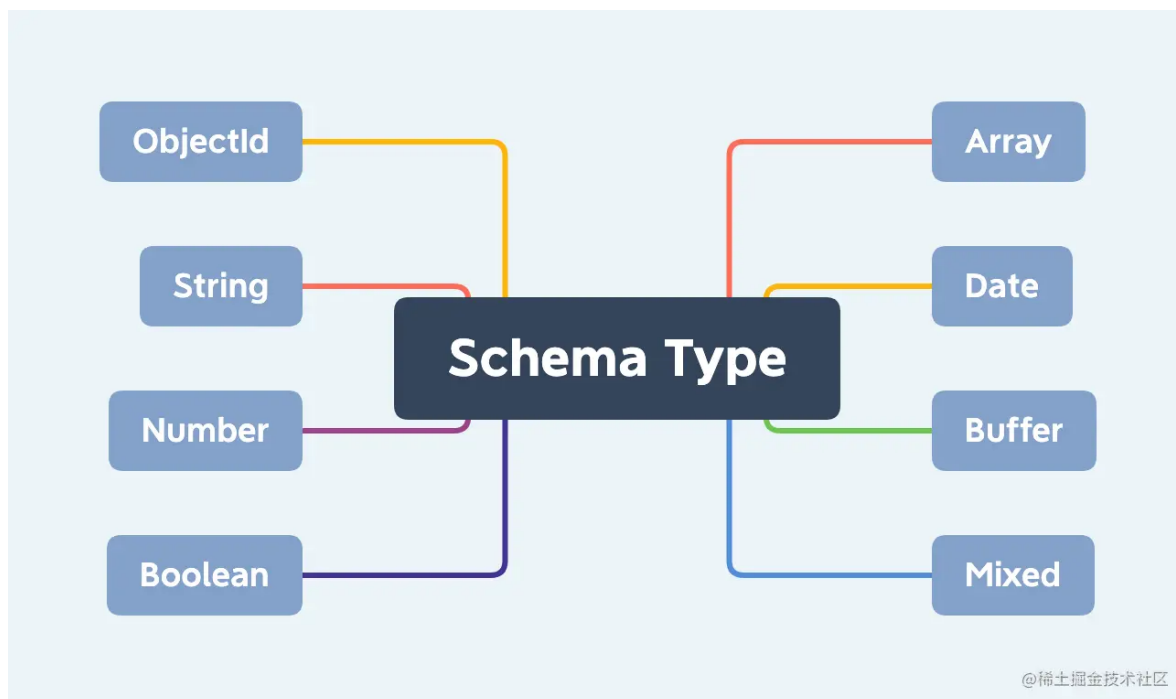
export {
    Product,
    ProductModel
};

```

点击查看[源码](#)。

其中每个产品数据都使用 `type` 定义数据类型，其类型值只能使用以下有效类型。

`String`、`Number`、`Boolean`、`Array` 和 `Date` 为原生数据类型，无需引用可直接使用。`Buffer` 为 `Node` 的特有数据类型，可查看[Node Buffer](#)。`ObjectId` 与 `Mixed` 为 `mongoose` 定义的数据类型。`ObjectId` 表示主键，每个 `Schema` 都会默认配置该属性，键值为 `_id`，在数据入库时会自动创建。`Mixed` 表示混合类型，可认为是引用类型的对象 `Object`。



通过 `export` 导出 `Product` (产品模式)与 `ProductModel` (产品类型), 就能在路由实例中自由操作 `MongoDB` 了。

定义接口

示例是一个 小型接口系统 , 仅仅是为了配合本课程讲述数据库的搭建与操作, 所以在代码形式上更多是为了确保搭建好的数据库的可用性与稳定性, 因此只编写了 产品的增删改查四个接口供你学习, 对于更多其他需求的接口, 可根据本章学习的内容自行编写。

新增产品

使用 `koa-router` 创建路由实例, 该实例提供 `get()` 与 `post()` 定义接口类型。两个方法需传入两个参数, 第一个是 接口路径 , 第二个参数是 上下文 。

```
import KoaRouter from "koa-router";
import { AsyncTo } from "@yangzw/bruce-us/dist/node";

import { Product, ProductModel } from "../../models";
import { CheckData } from "../../utils/setting";

const Router = KoaRouter();

Router.post("/mall/product/create", async ctx => {
  // 接口逻辑
}
```

js

```
});
```

```
export default Router;
```

接口路径的前缀在开发环境与生产环境可能不尽相同。在开发环境中为了方便可能需省略前缀 `/mall`，在生产环境中为了区分不同系统就必须增加前缀。

在上线前会确认开发环境与生产环境的 接口URL。

- 开发环境：`http://127.0.0.1/product/xyz`
- 生产环境：`https://api.yangzw.vip/mall/product/xyz`

基于上述确定情况，可用一个变量控制该前缀。在 `app.config.js` 加入以下内容。

```
export default {js  
  // ...  
  publicPath: env.NODE_ENV === "dev" ? "" : "/mall"  
};
```

接着开发第一个接口：**新增产品**。新增产品需用到几个参数，分别是 `brand`、`logo`、`description`、`name` 和 `origin`，无需 `createtime` 是因为可在数据入库时自动创建时间。

在开发接口时，需做到以下细节才能保障数据的准确性、安全性、可靠性和稳定性。

- ☑ 校验全部字段是否为空
- ☑ 校验全部字段是否符合正则
- ☑ 根据某个字段判断文档是否存在
- ☑ 读写数据库

```
Router.post(`${AppConfig.publicPath}/product/create`, async ctx => {js  
  const params = ctx.request.body;  
  // 校验全部字段是否为空  
  if (!CheckData(params, 5)) {  
    ctx.body = { code: 300, msg: "产品信息都不能为空" };  
    return false;  
  }  
  // 校验全部字段是否符合正则  
  const checkMsg = Object.entries(params).reduce((t, v) => {  
    const { match, msg } = Product[v[0]];
```

```
        return !t && !match.test(v[1]) ? { code: 400, msg } : t;
    }, "");
    if (checkMsg) {
        ctx.body = checkMsg;
        return false;
    }
    // 判断产品是否存在
    const [err1, res1] = await AsyncTo(ProductModel.findOne({ code: params.code }));
    if (err1) {
        ctx.body = { code: 400, msg: "新增产品失败" };
        return false;
    }
    if (res1) {
        ctx.body = { code: 400, msg: "当前产品已存在" };
        return false;
    }
    // 新增产品
    const [err2, res2] = await AsyncTo(ProductModel.create(params));
    if (!err2 && res2) {
        ctx.body = { code: 200, data: res2, msg: "新增产品成功" };
    } else {
        ctx.body = { code: 400, msg: "新增产品失败" };
    }
});
```

点击查看[源码](#)。

若使用 `post()` 可通过 `ctx.request.body` 获取入参，若使用 `get()` 可通过 `ctx.request.query` 获取入参。

剩余的 [删除产品](#)、[更新产品](#) 和 [读取产品](#) 也可用同样套路处理，在此不深入讲述了。

- [删除产品](#)：点击查看[源码](#)
- [更新产品](#)：点击查看[源码](#)
- [读取产品](#)：点击查看[源码](#)

调试：测试接口可行性

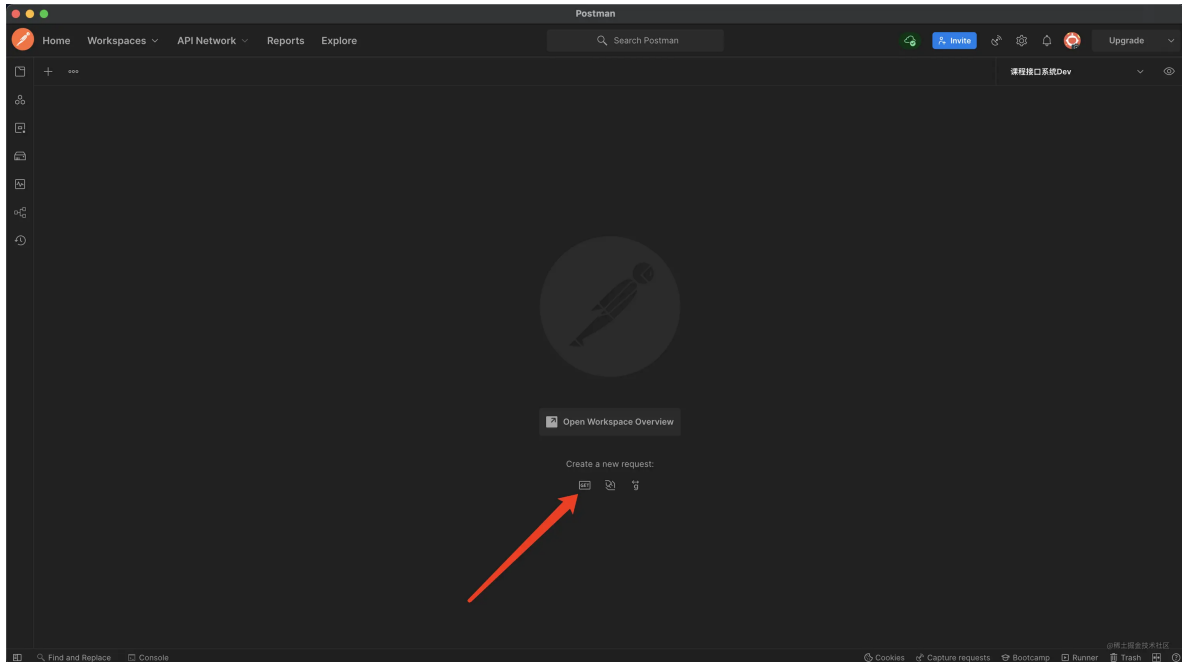
[接口系统](#) 运行起来，那就要测试每个接口的可行性，确保接口能正常使用。

Postman是一款功能强大用于测试HTTP请求的接口调试工具。虽然不能像 [QA](#) 那样用得很精，但必须知道如何创建与调试接口请求。

打开 **Postman** 的[下载地址](#)，根据系统选择适合的安装包再下载。

创建请求

安装完毕打开 **Postman**，点击 **Get** 按钮创建请求。

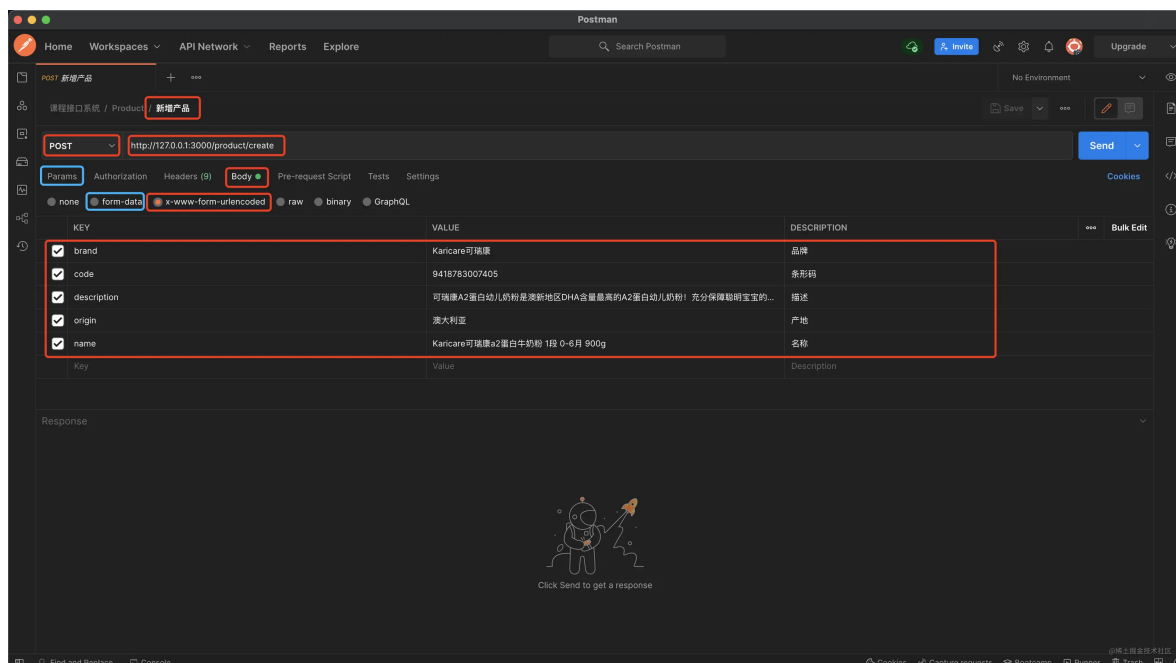


配置请求

在以下圈住的位置输入准确的参数信息，从左到右从上往下依次讲述。

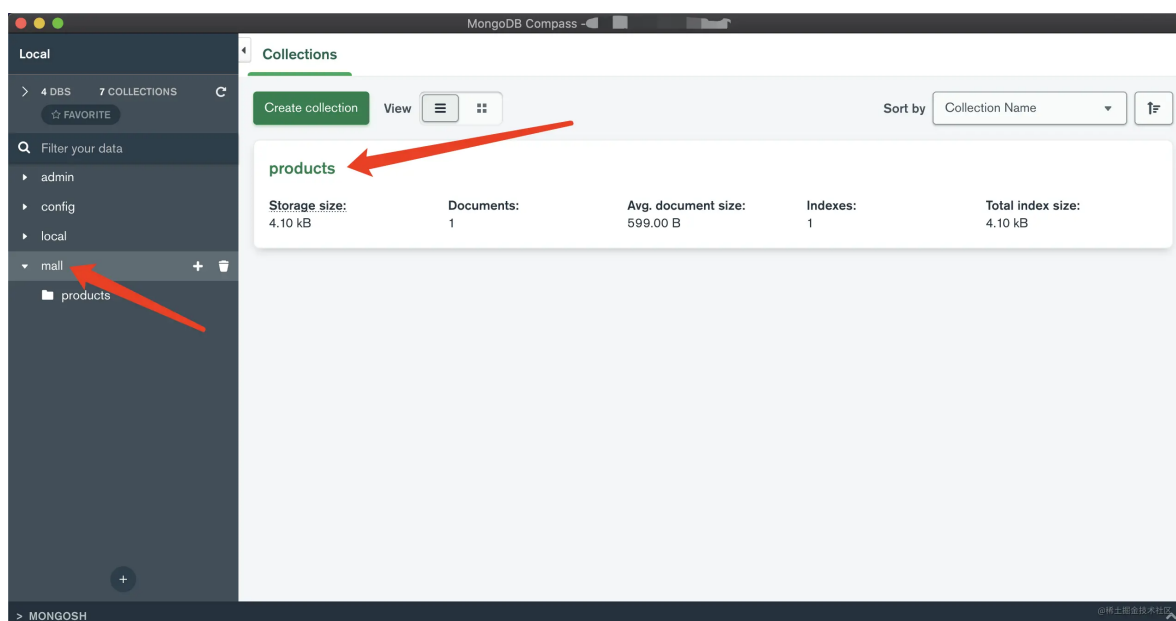
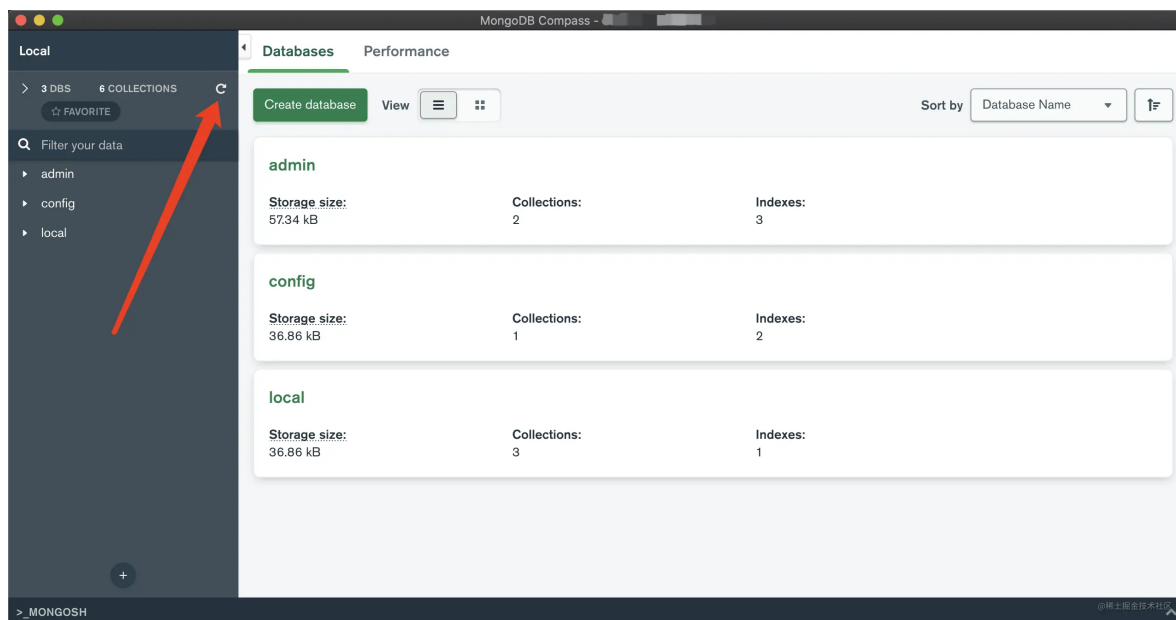
- **请求名称**：简要说明接口功能，可用文件夹分类
- **请求类型**：包括15种请求类型，很多情况选择 **GET** 或 **POST**
- **请求地址**：完整的接口请求路径
- **Params**：请求参数，用于 **GET** 入参，对应以下表格的数据
- **Body**：请求体，用于 **POST** 入参，对应以下表格的数据

Body 可选 **none/form-data/x-www-form-urlencoded** 等，无参数选 **none**，普通入参选 **x-www-form-urlencoded**，表单入参选 **form-data**。



输入完毕点击 **Send**，输出以下信息表示请求成功。

请求成功会在数据库中记录该文档。打开 **Compass**，点击刷新按钮，就能看到 **mongoose** 自动为 **MongoDB** 创建了一个数据库 **mall**，其中又创建了一个集合 **products**。**MongoDB** 有一个很好的功能就是存在某个数据库则使用它，不存在则自动创建。



回顾上述代码，**mall数据库** 是通过以下代码自动创建的。

```
connect(`mongodb://${username}:${password}@${host}:${port}/mall`, { ... });
```

js

products集合 是通过以下代码自动创建的。为何定义 **Model** 的名称是 **product** 却生成 **products**？因为在创建时 **mongoose** 会动态分析名称的词法，对不是复数的单词会追加 **s** 或 **es** 的后缀，所以在使用 **model()** 定义一个模型时，建议采用单词的单数形式。

```
model("product", new Schema(Product, { versionKey: false }));
```

js

使用相同方法在 **Postman** 中测试其余三个接口，若全部通过就表示本地环境运行的接口可行性测试通过。



部署：发布到服务器

上述都在本地环境部署，因此也只能在本地环境运行 **接口系统**。若要在外网通过 **HTTPS** 访问 **接口系统**，还需将 **Node服务** 发布到服务器环境中。

部署服务

通过第6章的方式在服务器中配置一个全新的 **api.yangzw.vip** 二级域名，该域名用于托管所有 **接口系统**。

若托管的 **接口系统** 有三个，分别是 **商城接口系统mall**、**博客接口系统blog** 和 **简历接口系统resume**，那应以前缀区分它们。

- 商城接口系统：**https://api.yangzw.vip/mall/xyz**，**Node服务** 监听端口为 **3000**
- 博客接口系统：**https://api.yangzw.vip/blog/xyz**，**Node服务** 监听端口为 **3001**
- 简历接口系统：**https://api.yangzw.vip/resume/xyz**，**Node服务** 监听端口为 **3002**

打开 **FTP工具**，在 `/www/server` 目录中创建 `data-base` 文件夹，把本地的 `data-base` 文件夹中除了 `node_modules` 文件夹的其他内容上传上去。执行 `npm i` 安装依赖。

打开 **CMD工具**，登录服务器。在 `/etc/nginx/conf.d` 目录中创建 `api.yangzw.vip.conf` 文件，执行 `vim /etc/nginx/conf.d/api.yangzw.vip.conf`，加入以下内容。

```
server {                                                                 conf
    server_name api.yangzw.vip;
    location /mall {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr;
        proxy_set_header X-Forwarded-Proto https;
    }
    location /blog {
        proxy_pass http://127.0.0.1:3001;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr;
        proxy_set_header X-Forwarded-Proto https;
    }
    location /resume {
        proxy_pass http://127.0.0.1:3002;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr;
        proxy_set_header X-Forwarded-Proto https;
    }
}
```

执行 `certbot --nginx`，选择二级域名 `npm.yangzw.vip`，输出以下信息表示配置成功。

```
Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/api.yangzw.vip/fullchain.pem
Key is saved at:         /etc/letsencrypt/live/api.yangzw.vip/privkey.pem
This certificate expires on 2022-08-09.
These files will be updated when the certificate renews.
Certbot has set up a scheduled task to automatically renew this certificate in the backg
```

Deploying certificate

```
Successfully deployed certificate for api.yangzw.vip to /etc/nginx/conf.d/api.yangzw.vip
Congratulations! You have successfully enabled HTTPS on https://api.yangzw.vip
```

每次新开端口都需在服务器中配置安全组，可回看第7章的 [个人官网-安全组配置](#) 那部分内容。这次增加 3000 端口，那在安全组中配置一个 3000 端口。

最后执行 `nginx -t` 验证 Nginx 配置，再执行 `nginx -s reload` 重启 Nginx 进程。执行 `npm run deploy` 启动服务。

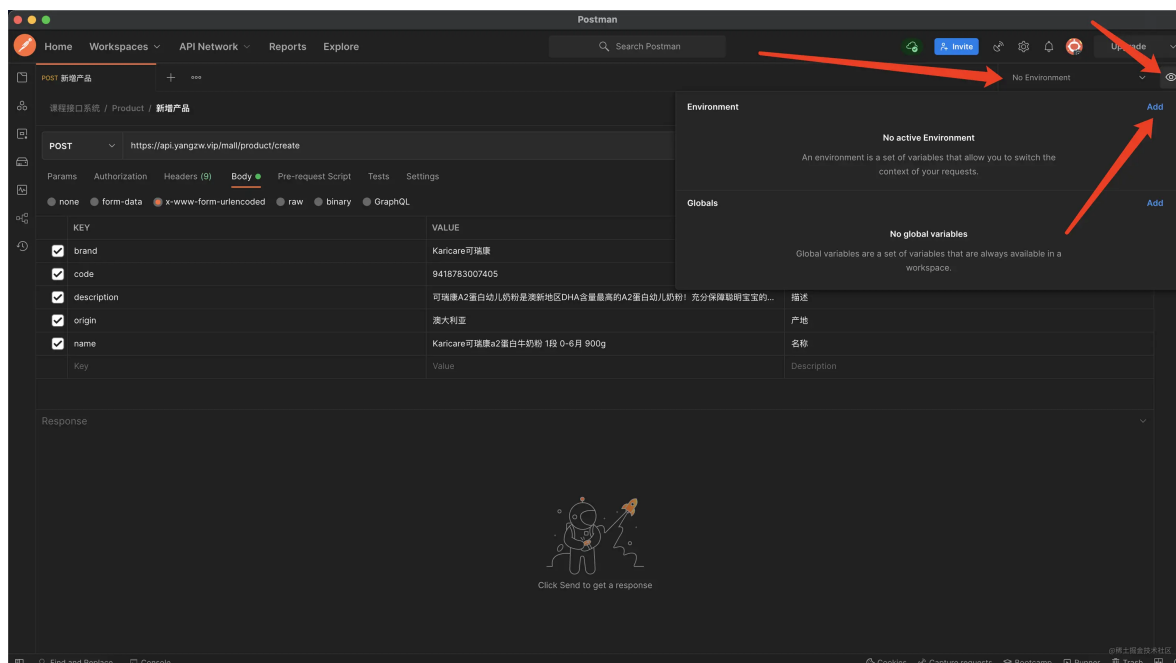
调试接口

回到 Postman 的新增产品的接口，将 `http://127.0.0.1:3000` 改成 `https://api.yangzw.vip/mall`。

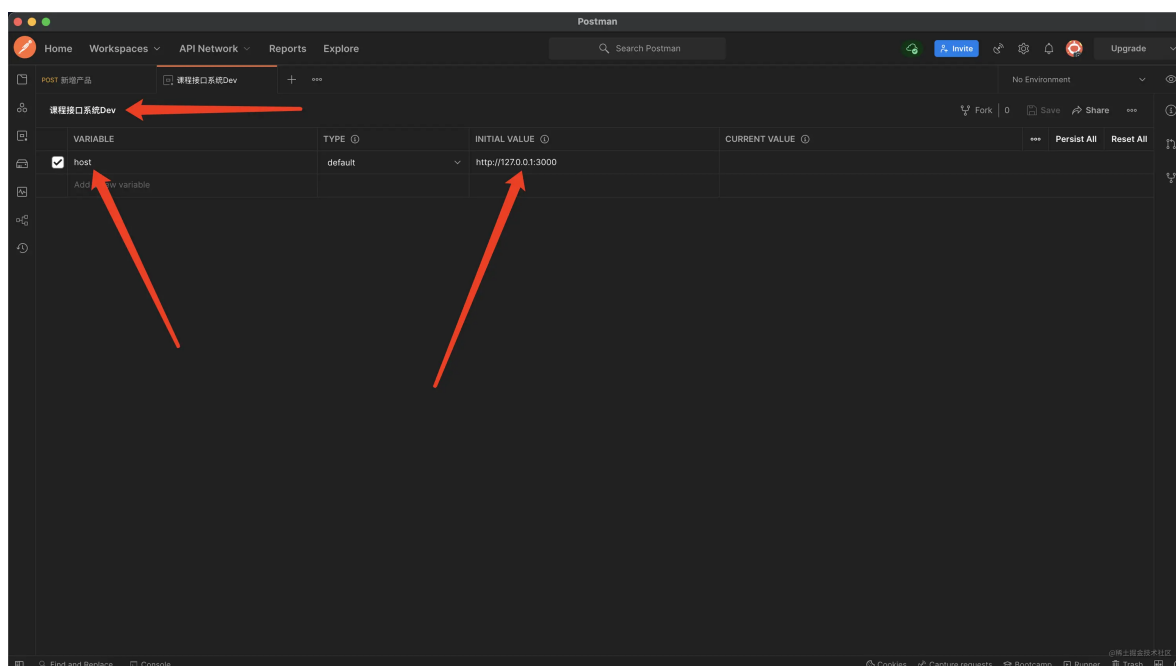
输入完毕点击 `Send`，输出以下信息表示请求成功。若不成功请检查上述步骤，看看是哪里遗漏了。

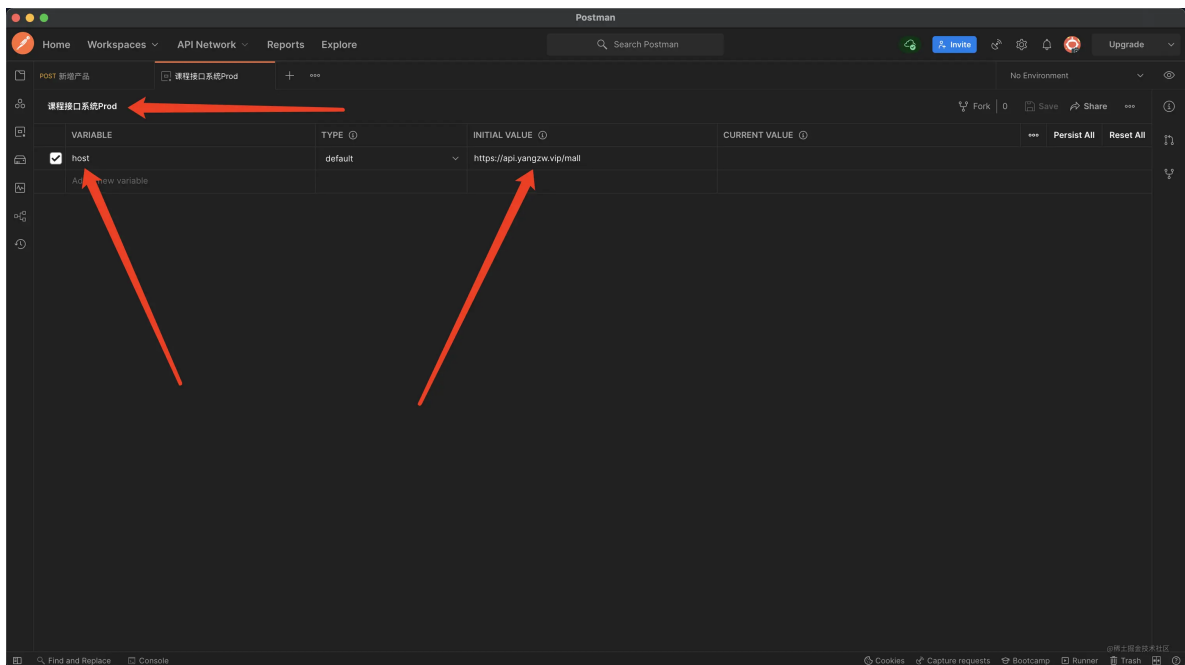
在默认情况下，开发环境与生产环境的 [接口域名](#) 都不会一样，但 [接口路径](#) 却一样。在 Postman 中遇到不同环境调试接口，那得把改 [接口域名](#) 改来改去了。

在 Postman 右上角的环境变量中选择 `No Environment`，点击右边的 [查看](#) 图标，再点击 `Add` 按钮。



输入 **全局环境** 的名称，设置当前环境中的变量内容。分别创建 **开发环境** 与 **生产环境** 的变量。





创建完毕就能愉快地调试不同环境中的接口实例了。

结语

说实话，很多公司企业都不会使用 **Node** 开发 接口系统，顶多就使用 **Node** 开发一个接口中转服务，以实现接口数据的格式化或过滤。

当然掌握多一个技能就是多一份竞争力，对于像阿里这样的大企业，使用 **Node** 开发 接口系统 还是很少有的。建议你有时间可尝试使用 **Node** 与 **MongoDB** 开发一个 接口系统，尝试新的玩法。

本章内容到此为止，希望能对你有所启发，欢迎你把自己的学习心得打到评论区！

✅ 示例项目：[fe-engineering](#)

✅ 正式项目：[bruce](#)

留言

输入评论（Enter换行，Ctrl + Enter发送）

发表评论

全部评论 (7)



ZH_

10天前

封装接口系统时若出现引入报错，需要package.json加入"type": "module",或者index后缀mjs，或者使用cjs写index.js

点赞 回复

NewName

前端CV

2月前

想让date-base服务跑起来，得进入到/www/server/data-base目录然后npm run deploy，这样用 api.yangzw.vip 调试接口才好使;但是 ctrl+c 了就GG了。怎么办呢?

解决办法在下一节，老师好棒啊

点赞 1

JowayYoung (作者) 2月前

适当的铺垫

点赞 回复

NewName

前端CV

2月前

温馨提示，下载Compass时如果win 64 则 选择1.32.3-win32-x64，亲测好使~

点赞 回复



luobob

front-end engeneer ...

4月前

请问一下脑图是用什么软件画的呢

点赞 2

JowayYoung (作者) 4月前

xmind，需要破解的

点赞 回复



luobob 回复 JowayYoung 4月前

好嘞

“xmind，需要破解的”

点赞 回复

